JAVA SIMPLE ENGINE

A 2D arena demonstrating six coordinated design patterns

Software Design Patterns
Team technical report

Sabirzhanov Emil — Bridge and Decorator
Baktiyarova Aruzhan — Factory Method and Abstract Factory
Roziyeva Yasmina — Strategy and Observer

2 November 2026
Version 0.1.0 — consolidated implementation
Draft for team review

A Java desktop runtime and arena example with interchangeable graphics, composable visual layers, themed object creation, replaceable enemy behavior and independent event observers. This report describes the implemented application and records its verified limits.

1 Introduction and objectives

The project addresses a small game-engine problem: how can object creation, enemy decisions, event displays and graphics evolve without forcing changes throughout the application? Java Simple Engine (JSE) separates these responsibilities and brings them together in an interactive 2D arena. The implementation uses Java 17 release bytecode, Swing and Java2D, with no third-party production libraries.

The course requires six distinct patterns to function in one coherent project, with two pattern responsibilities per contributor [1,2]. The patterns use the vocabulary of Gamma, Helm, Johnson and Vlissides [3]. This report follows the code in the consolidated repository rather than treating planned classes as implemented features.

Pattern	Team responsibility	Visible demonstration
Bridge	Emil	F1 changes Filled/Wireframe without recreating entities.
Decorator	Emil	F2/F3 compose outline and live Player health layers.
Factory Method	Aruzhan	Three spawner subclasses create Player, Enemy and Pickup categories.
Abstract Factory	Aruzhan	T creates a coherent Forest or Space product family.
Strategy	Yasmina	Enemies Patrol, Chase or Flee; Tab/B replace behavior while running.
Observer	Yasmina	HUD and log follow events; L removes/recreates the log independently.

Scope and success criteria

The deliverable is an executable JAR, reproducible source and a report containing real code excerpts, screenshots and UML. The application must support movement, collision consequences, a terminal outcome, reset/theme/menu transitions and a standalone rendering showcase. Success includes preserving entity state during visual changes and closing scene-owned resources during transitions.

3D rendering, networking, persistence, audio and a level editor are outside the implementation. Diagnostic spawning and a stronger bounded deferred-event protocol are listed as further work; neither is required to explain the six patterns currently demonstrated.

2 Architecture and runtime

The composition root loads validated resource profiles and preloads both sprite families before opening the window. Core knows only Scene, Renderer, InputSource and EngineHost. GameScenes owns navigation and shared visual choices; each ArenaRun owns a fresh World, rule state, EventBus and concrete observers. No mutable global engine or shared World is used.

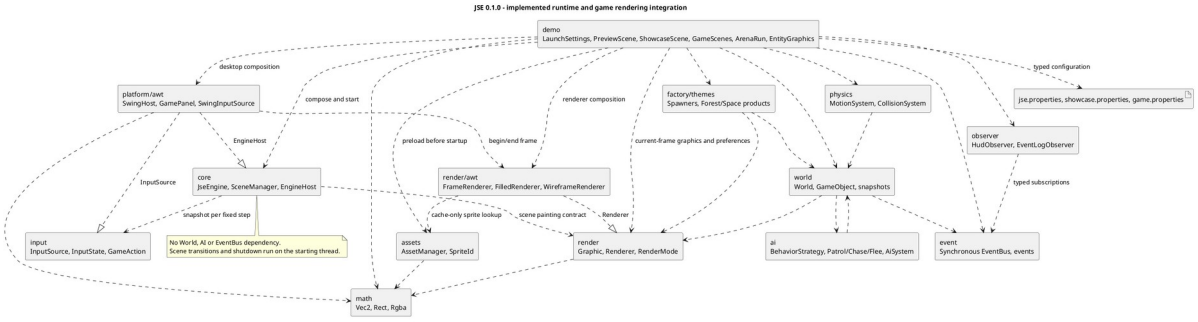


Figure 1. Implemented runtime and game composition. Core has no dependency on the game model or EventBus.

Swing Timer callbacks execute on the event dispatch thread [4]. A timer pulse pumps the engine, while FixedStepClock supplies fixed updates with bounded catch-up. Scene transitions cancel remaining catch-up steps and clear input. GamePanel creates a frame-local graphics copy; renderers release their Graphics2D reference in endFrame. Input aliases and pressed actions are tracked separately.

Maven Wrapper pins the build-tool version [5]. The repository selects Maven 3.9.9, compiles release 17 bytecode and packages its resources into target/jse-demo.jar. JUnit 5.10.2 is a test dependency; the application itself uses the JDK only.

3 Factory Method and Abstract Factory

Factory Method: choose a product category

Factory Method delegates creation to a subclass operation. In JSE, EntitySpawner is the creator and final spawn owns request checking, product category/family validation and queued World registration. PlayerSpawner, EnemySpawner and PickupSpawner override createObject for their corresponding category. The operation returns a real model object with a stable World-assigned identity after registration.

Listing 1. EnemySpawner supplies the category-specific creation operation. (src/main/java/jse/factory/EnemySpawner.java:14)

```
protected Enemy createObject(SpawnRequest request) {
    return factory.createEnemy(request);
}
```

Abstract Factory: choose a coherent theme

Abstract Factory creates related products through a shared interface. GameObjectFamilyFactory exposes createPlayer, createEnemy and createPickup. ForestFactory and SpaceFactory return six concrete products with matching family IDs and SpriteGraphic resource paths. Each factory validates the requested category size against injected settings. T selects a different family and starts a new run; spawner classes and game rules stay the same.

Listing 2. The selected family factory creates its themed Enemy and appearance. (src/main/java/jse/factory/ForestFactory.java:49)

```
public Enemy createEnemy(SpawnRequest request) {
    requireSize(request, settings.enemy().size());
    Graphic graphic = new SpriteGraphic(renderer, sprites.enemy());

    return new ForestEnemy(request, graphic, settings.enemy());
}
```

Question	Responsible structure
Which category is created?	The concrete EntitySpawner subclass and createObject override.
Which related family is created?	The injected GameObjectFamilyFactory implementation.
Who assigns IDs and activates objects?	World, through queued registration and commitChanges.

These are separate creation axes. Factories assemble products, spawners apply the common creation workflow, and World owns identity and lifecycle. Category settings supply size, speed, health and pickup value rather than copying constants into six product constructors.

4 Creational UML and object lifecycle

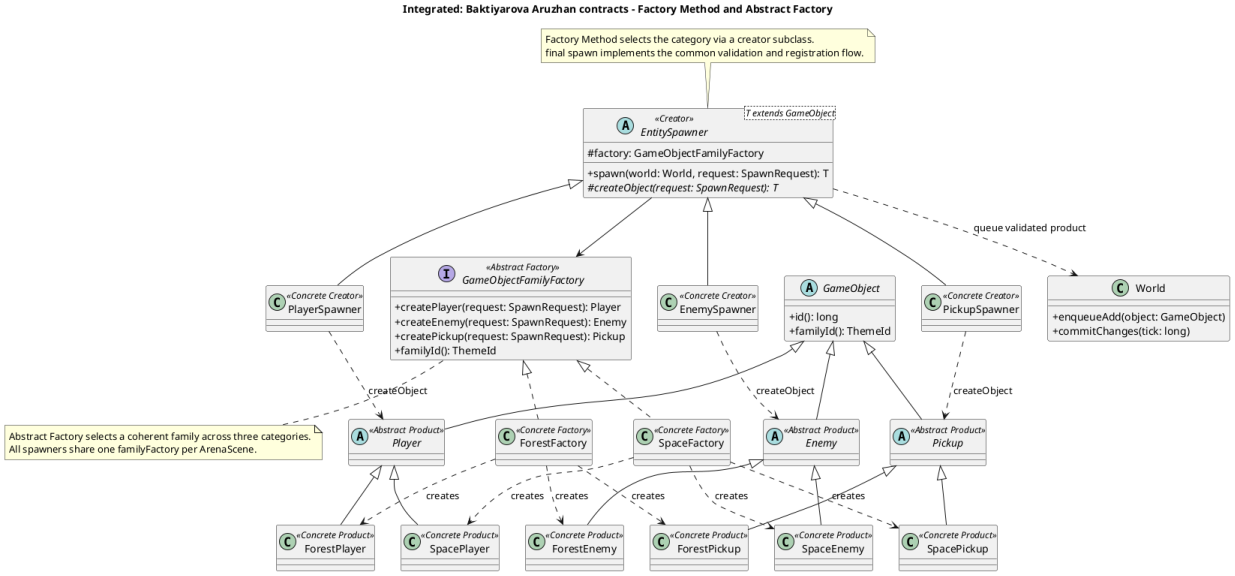


Figure 2. Actual category creators, family factories and six related products.

A spawn is initially pending: it has an assigned positive ID but is absent from active lookups until commitChanges. The spawn snapshot includes pending objects so callers can reserve occupancy. Removal makes an object unavailable immediately and changes storage at the next commit. A cancelled pending addition consumes its ID without emitting a spawn/removal lifecycle pair.

FamilyFactoryTest covers both themes and all three categories, injected values, size rejection and queued registration through the three spawners. WorldTest separately covers stable IDs, capacity/family/bounds validation, immutable snapshots, cancellation and safe mutation during iteration.

5 Bridge — Emil's graphics abstraction

Bridge separates an abstraction from interchangeable implementations. Graphic is the abstraction; RectangleGraphic and SpriteGraphic refine it. Renderer is the implementor interface, implemented by FilledRenderer and WireframeRenderer through the Java2D frame boundary. Each graphic delegates drawing operations to its Renderer rather than owning a Java2D context.

Role	Actual code
Abstraction	jse.render.Graphic
Refined abstractions	RectangleGraphic, SpriteGraphic
Implementor	Renderer
Concrete implementors	FilledRenderer, WireframeRenderer
Integration client	EntityGraphics and the scene render(Renderer) contract

Listing 3. SpriteGraphic delegates its appearance to the implementor. (src/main/java/jse/render/SpriteGraphic.java:15)

```
public void draw(Rect bounds) {
    renderer.drawSprite(sprite, bounds);
}
```

Runtime replacement and frame safety

F1 changes the host's active renderer. EntityGraphics receives the Renderer supplied for the current frame, propagates it into every current graphic and draws the object's existing bounds. The World, entity ID, position, health and theme are preserved. Core, platform and graphics contracts remain separate; a drawing context is not retained after a frame.

Both concrete implementations use Java2D. Filled draws actual sprite images and filled rectangles; Wireframe draws bounds and diagonals. They are two production output styles, rather than a production renderer paired with a test spy. Game wireframes omit resource-path labels to keep the arena readable; the standalone showcase can retain those labels.

RendererImageTest and EntityGraphicsTest compare real BufferedImage pixels before and after renderer replacement. The earlier renderer's frame has already ended, so a stale reference in any wrapped layer would fail drawing. This protects the key Bridge behavior rather than only asserting that an implementation class exists.

6 Decorator — Emil’s composable visual layers

Decorator adds optional responsibilities by wrapping a component. GraphicDecorator extends Graphic and retains a wrapped Graphic. Drawing first delegates to the wrapped component, then paints its own layer. The same component interface allows an outline and health bar to be used separately or together.

Listing 4. Delegation keeps the original appearance before adding a layer. (src/main/java/jse/render/GraphicDecorator.java:21)

```
public final void draw(Rect bounds) {
    Objects.requireNonNull(bounds, "bounds");
    wrapped.draw(bounds);
    drawLayer(bounds);
}
```

Listing 5. Renderer replacement propagates through the entire wrapped chain. (src/main/java/jse/render/GraphicDecorator.java:15)

```
public final void setRenderer(Renderer renderer) {
    super.setRenderer(renderer);
    wrapped.setRenderer(renderer);
}
```

Composition, live health and lifecycle

OutlineDecorator uses four configured-width bands outside the object’s bounds. HealthBarDecorator reads the actual Player HealthView on each draw, checks that maximum health is positive and clamps the current/max ratio. A zero ratio paints only the background. Both layers change appearance without changing collision geometry.

GraphicLayers composes Outline and then HealthBar from the immutable baseGraphic. EntityGraphics caches the applied visual choices, rebuilds only when those choices change, and restores retired objects to their base graphic. Only Player receives a health layer in Arena; non-player entities remain unaffected by the health toggle.

The tests use real damage and pixel assertions, exercise 200 recompositions without chain growth, verify independent layers and check renderer propagation. Scene exit closes EntityGraphics and the run’s observers and bus. This keeps a visual option from accumulating objects or interfering with future runs.

7 Rendering UML and pattern interaction

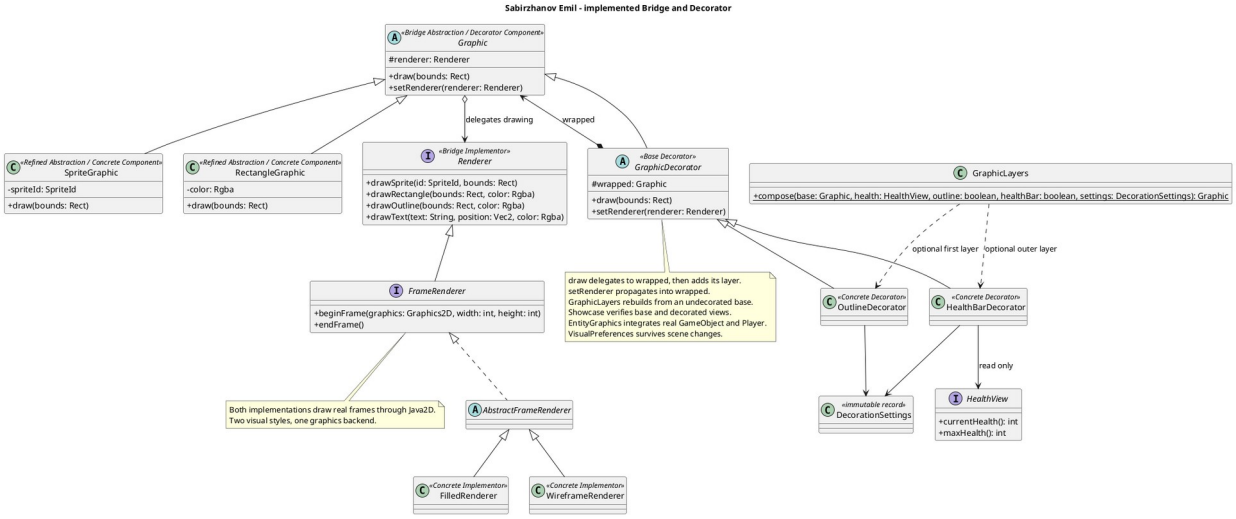


Figure 3. Bridge and Decorator share Graphic as their abstraction/component.

The two structural patterns serve different decisions. Bridge chooses how a graphic is rendered; Decorator chooses which additional layers surround it. F1 therefore works independently of F2/F3. The same sprite can be plain, outlined, health-decorated or combined under either production renderer.

The standalone Showcase compares a base and decorated graphic and displays a configured partial health example. Arena uses the same graphic contracts with real model state and actual damage. Keeping Showcase independent of World makes the graphics demonstration available even when explaining only Emil's two patterns.

8 Strategy — replaceable enemy decisions

Strategy makes an algorithm interchangeable behind a common contract. Enemy holds a BehaviorStrategy. Idle, Patrol, Chase and Flee calculate a velocity from an Enemy, one immutable WorldView snapshot and dt. Strategies return intent; MotionSystem alone integrates and clamps positions.

Role	Actual code
Strategy contract	BehaviorStrategy.calculateMovement; desiredVelocity compatibility alias
Context	Enemy.behavior and setBehavior
Concrete strategies	IdleBehavior, PatrolBehavior, ChaseBehavior, FleeBehavior
Client	AiSystem.update and changeBehavior
Runtime selection	ArenaRun initializes Patrol/Chase/Flee; Tab/B select and replace

Listing 6. The AI client applies each enemy's own strategy before motion. (src/main/java/jse/ai/AiSystem.java:23)

```
public void update(World world, WorldView view, double dt) {
    Objects.requireNonNull(world, "world");
    Objects.requireNonNull(view, "view");
    world.forEachActive(object -> {
        if (object instanceof Enemy enemy) {
            Vec2 velocity = enemy.behavior().calculateMovement(enemy, view, dt);
            enemy.setVelocity(Objects.requireNonNull(velocity, "strategy returned null velocity"));
        }
    });
}
```

Patrol has per-instance waypoint state and reduces speed on the final step so it does not overshoot. Chase stops without a target or at zero distance. Flee stops outside its configured radius and chooses a stable direction on exact overlap. Replacing a behavior preserves entity ID, position and family and emits BehaviorChanged once for a real replacement.

The scene cycles the demonstration options outside Enemy. A new strategy implementation does not require editing Enemy or the generic AI update loop; adding that option to the demonstration chooser is a separate composition decision. BehaviorTest covers concrete algorithms and replacement, while ArenaBehaviorsTest proves the strategies are called by the real run.

9 Observer — independent event displays

Observer notifies interested consumers without coupling publishers to their displays. EventBus routes exact event classes to typed EventListener callbacks and returns Subscription handles. World publishes lifecycle events, ArenaRules publishes rule events, and AiSystem publishes behavior replacements.

Listing 7. HUD state is initialized from Player health and then follows typed events. (src/main/java/jse/observer/HudObserver.java:22)

```
public HudObserver(EventBus events, Player player) {
    Objects.requireNonNull(events, "events");
    this.health = Objects.requireNonNull(player, "player").currentHealth();
    subscriptions.add(events.subscribe(ItemCollected.class, e -> score = e.newScore()));
    subscriptions.add(events.subscribe(HealthChanged.class, e -> health = e.newHealth()));
    subscriptions.add(events.subscribe(GameFinished.class, e -> result = e.result()));
}
```

HudObserver supplies displayed score and health and records the result. EventLogObserver retains the latest configured number of lines from six event types. ArenaRun creates both and closes their subscriptions before closing its EventBus. The screen reads HUD score and health and reads the outcome heading from rule status. Rules keep their own authoritative gameplay state and never read HUD values.

Listing 8. L removes or recreates the log independently of the HUD. (src/main/java/jse/demo/ArenaRun.java:101)

```
public void setLogEnabled(boolean enabled) {
    if (closed) throw new IllegalStateException("Arena run is closed");
    if (enabled && log == null) log = new EventLogObserver(events, game.logCapacity());
    else if (!enabled && log != null) { log.close(); log = null; }
}
```

Disabling L stops log delivery; reenabling starts with an empty log and does not replay missed events. HUD delivery continues while the log is detached. This independence is checked through real collision health events as well as direct observer unit tests.

Actual dispatch semantics

The current bus normally publishes synchronously. Publications made inside a callback are queued and drained after the outer dispatch; they do not wait for a later engine tick. This report does not claim the stronger bounded, next-tick batching protocol described as a future target in the technical specification. Observer remains operational with its documented current semantics.

10 Behavioral UML and coordinated flow

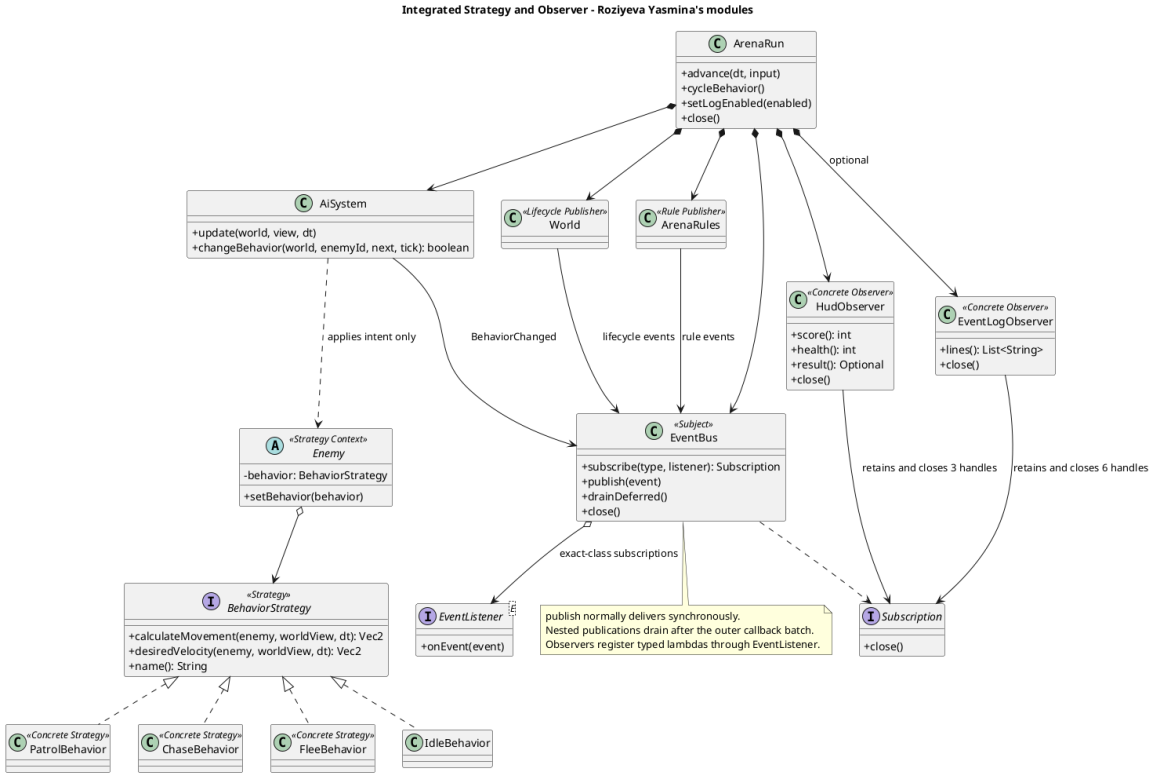


Figure 4. Actual Strategy clients and independently subscribed HUD/log observers.

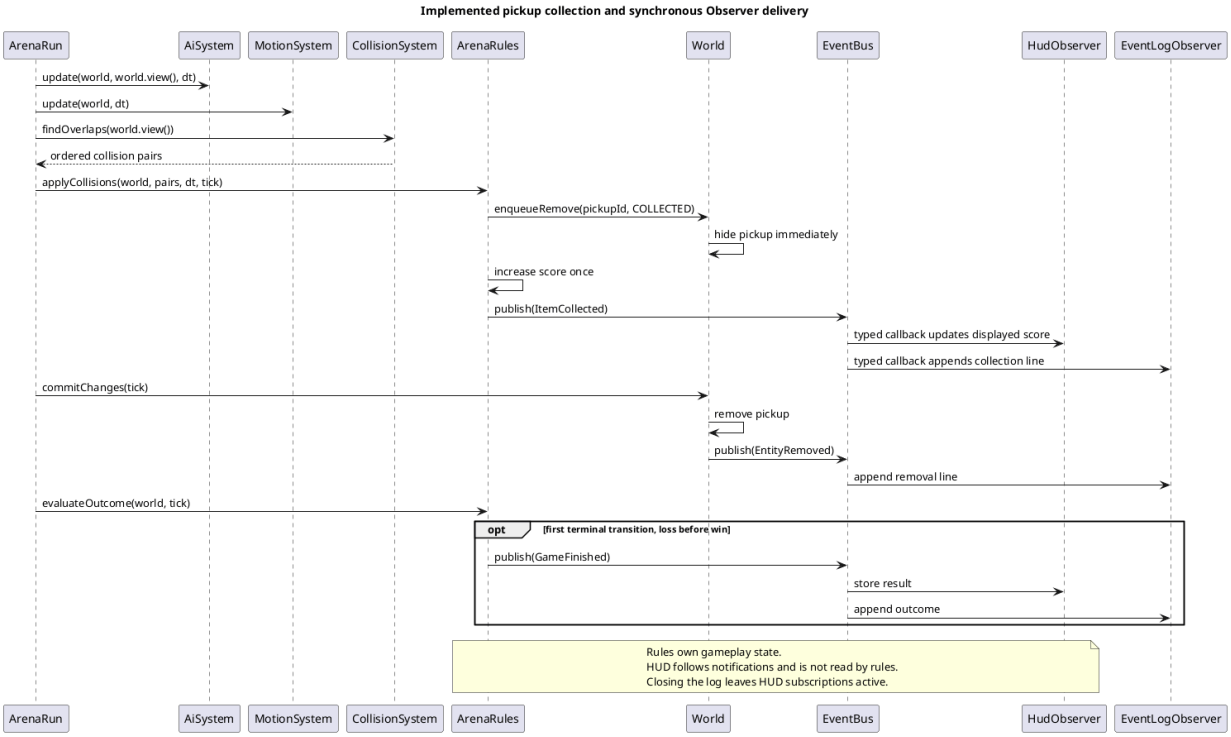


Figure 5. A real pickup interaction publishes rule and lifecycle events; HUD and log update through subscriptions.

11 Verification and practical limits

The consolidated suite passes 94 tests across 26 test classes, with no failures or errors and 0 skipped tests. The report generator reads these figures from the current Surefire XML results. Production and test sources compile for Java 17 release bytecode; verification ran on Linux with OpenJDK 25.0.3.

Area	Evidence
Engine/input/platform	RuntimeTest, InputStateTest, SwingInputSourceTest and GamePanelTest: fixed updates, pressed consumption, lifecycle failure/cleanup and frame ownership.
Bridge/Decorator/assets	RendererImageTest, GraphicDecoratorTest, EntityGraphicsTest and asset tests: actual pixels, current renderer, live health, bounded layers and preloading.
Creation/World	FamilyFactoryTest and WorldTest: six-product matrix, creators, family/size/capacity validation, queued lifecycle and immutable snapshots.
Strategy/Observer	BehaviorTest, ObserverTest and EventBusTest: behavior algorithms, runtime replacement, exact routing, independent unsubscribe and log capacity.
Real Arena wiring	ArenaBehaviorsTest and ArenaIntegrationTest: live AI, HUD/log events, cooldown, single collection, outcome priority and controls while paused.
Lifecycle repetition	GameSceneLifecycleTest: 20 actual engine route cycles and 60 old runs closed; observer tests also repeat subscribe/close cycles.
Desktop verification	tools/DesktopVerification.java: actual Swing window, frame captures, Tab/B/L, live collision health, renderer switching and 20 additional route cycles.

Reproduce the build

```
./mvnw clean verify
java -jar target/jse-demo.jar --example game
java -jar target/jse-demo.jar --example showcase
java -Djava.awt.headless=true -jar target/jse-demo.jar --help
```

The packaged JAR contains the three settings profiles, six sprites and their attribution and excludes JUnit. Windows and macOS were not tested. Installed Swing actions are the repeatable control proof; native Robot Enter is recorded separately by the desktop tool. Physical human keyboard delivery and every native OS shortcut are not claimed as verified.

12 SOLID and team responsibilities

Principle	Concrete application
Single Responsibility	AI selects velocity; MotionSystem moves; CollisionSystem detects; ArenaRules applies consequences; graphics render; observers display.
Open/Closed	A new strategy implements BehaviorStrategy; a new family implements GameObjectFamilyFactory; generic AI and spawners remain unchanged.
Liskov Substitution	Renderers obey frame lifetime contracts; concrete products preserve family/category identity; graphics and decorators accept the same bounds contract.
Interface Segregation	HealthView exposes only health; WorldView exposes immutable state; InputSource, Renderer and EngineControl have separate purposes.
Dependency Inversion	Core uses host/input/render contracts; spawners use the family-factory interface; rule publishers notify observers through EventBus.

Agreed pattern responsibilities

Contributor	Two primary patterns	Main implementation locations
Sabirzhanov Emil	Bridge; Decorator	jse.render, render.awt, assets, platform/core support, EntityGraphics and rendering showcases.
Baktiyarova Aruzhan	Factory Method; Abstract Factory	jse.factory, jse.theme.forest, jse.theme.space and the shared object categories.
Roziyeva Yasmina	Strategy; Observer	jse.ai, jse.event and jse.observer, coordinated through the Arena run.

This table records team responsibilities rather than asserting that only one person edited each module. Integration crossed module boundaries to connect contracts, validate the runnable system and prepare artifacts. Git history preserves the actual commit authors and the merge of each contributor's branch.

The final integration retains Aruzhan's family products and factories, Yasmina's AI/observer modules and Emil's runtime/rendering work. Factory request-size validation is retained during conflict resolution. Configuration profiles separate engine settings, showcase visuals and game values.

13 Application screenshots: render styles

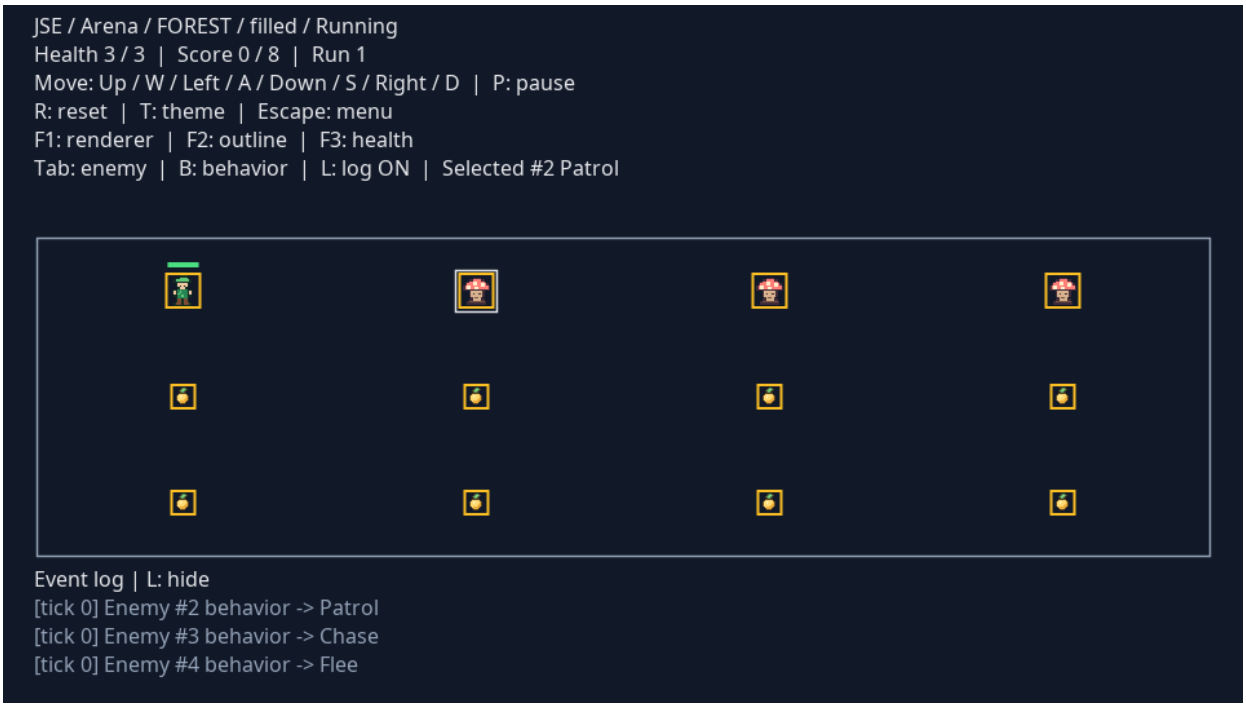


Figure 6. The unified Arena: themed objects, HUD values, selected enemy and independently observed log.

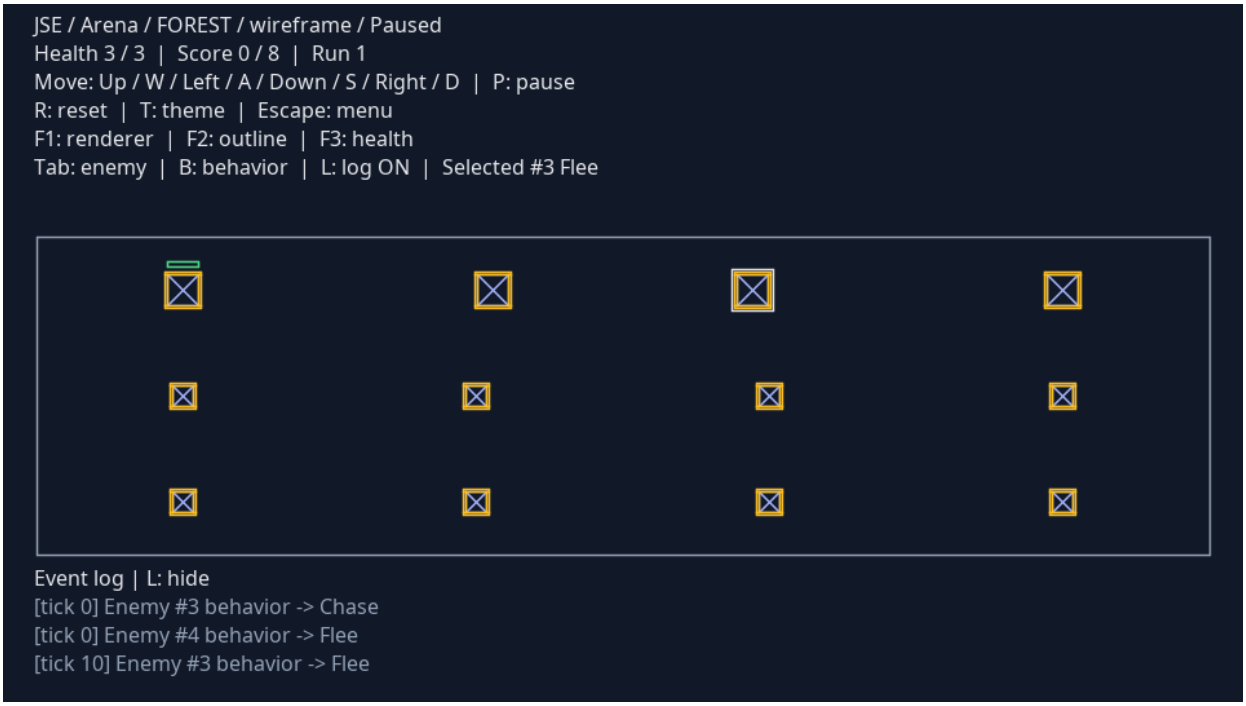


Figure 7. Bridge changes the actual output style while the paused model and selected strategy remain intact.

14 Application screenshots: behavior and observers

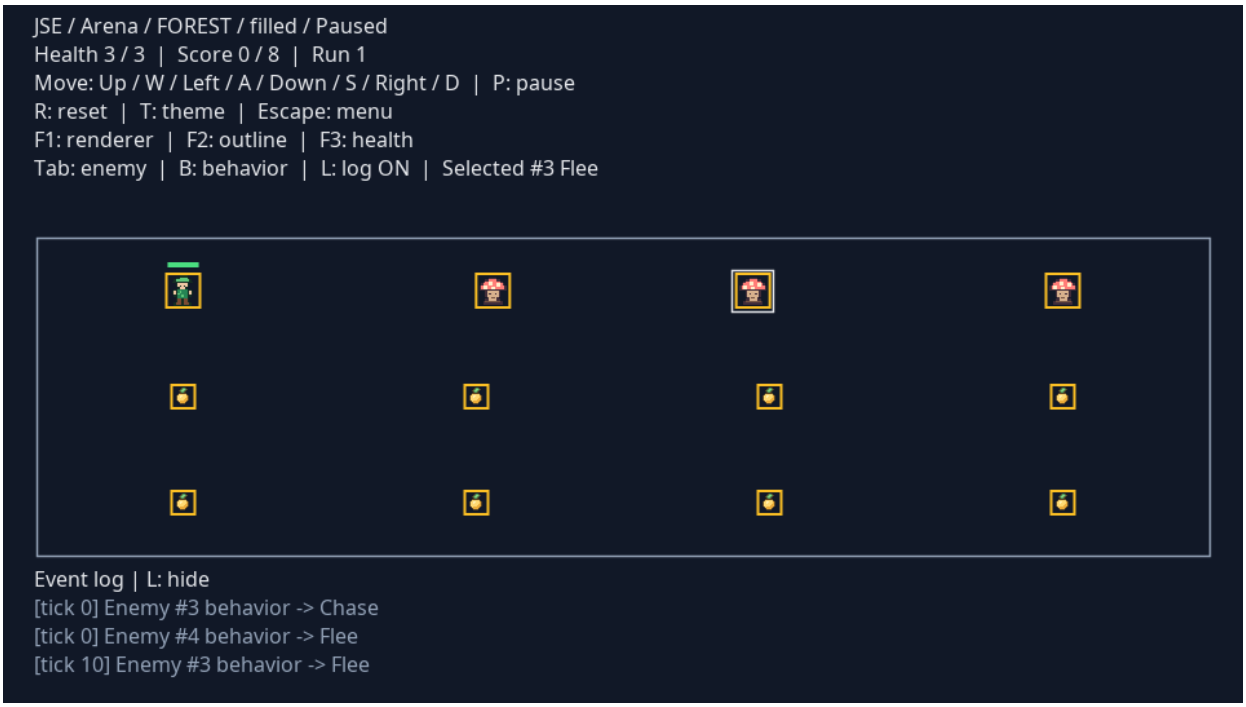


Figure 8. Tab selects an enemy and B replaces its strategy during pause; the log receives BehaviorChanged.

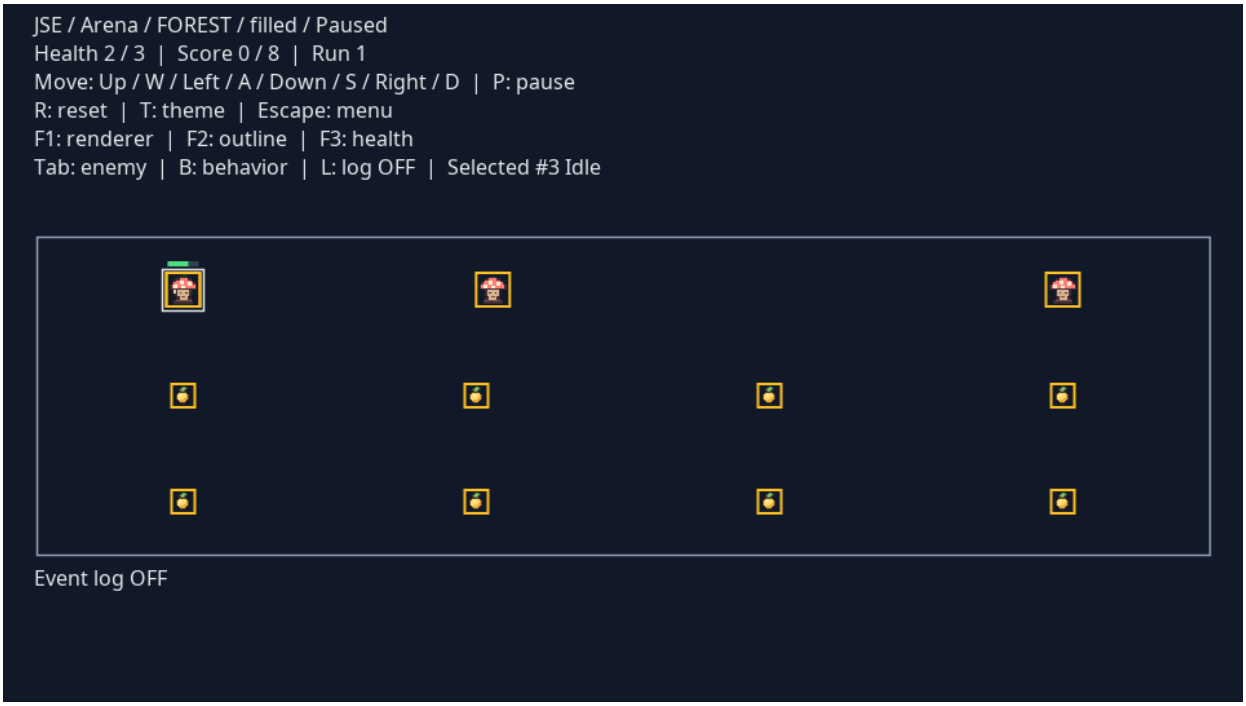


Figure 9. A controlled real collision reduces Player health and updates the HUD while the log is detached.

15 Conclusion, further work and defense

JSE demonstrates six distinct patterns in one executable application. Its creation workflow keeps category choice separate from theme choice. Its renderer abstraction works with composable visual layers. Enemy algorithms are replaceable, and the HUD/log receive events independently of the rules. Tests and desktop checks support these claims through observable model state, pixels and lifecycle behavior.

Further work

- Verify the packaged application on Windows/macOS and record native keyboard behavior on each supported platform.
- Add diagnostic spawning and a richer reference layout if these become product requirements.
- Evolve EventBus to an explicitly bounded deferred protocol with corresponding observer lifecycle tests; migrate legacy event classes together.
- Extend gameplay content, accessibility and renderer backends while retaining the current abstractions.

Suggested defense sequence

1. Run the packaged game and enter Arena. Pause while describing the architecture.
2. Use F1 to explain Bridge, then F2/F3 independently to explain Decorator.
3. Use T to recreate a different family; explain creators versus family factories.
4. Use Tab/B during pause, then resume to show Patrol, Chase and Flee motion.
5. Use L to detach the log. Show a health/score event still reaching the HUD, then reenable the log.
6. Show an outcome, R reset, return to Title and open Showcase with H.
7. Point to real class roles in the UML, code excerpts and automated verification evidence.

16 References and delivery notes

[1] Course material. Final Project Defense Requirements. Supplied PDF, first slide: six coordinated patterns, two per student, report structure and PDF/source-ZIP deliverables.

[2] Course material. Software Design Patterns Syllabus. Supplied PDF: project report and defense requirements. Local source materials are kept outside the public source archive.

[3] Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. ISBN 9780201633610. Publisher catalogue

[4] Oracle. Java SE 17 API Specification: javax.swing.Timer. Accessed 2 November 2026. Official Java API documentation

[5] Apache Software Foundation. Apache Maven Wrapper documentation. Accessed 2 November 2026. Official Maven Wrapper documentation

[6] JSE source repository. Java sources, tests, configuration, original sprite attribution and PlantUML sources. Project repository

Report and source artifacts

The editable report is docs/report/JSE-Technical-Report.docx; the submission-format version is JSE-Technical-Report.pdf. Report prose is stored in report-content.json and rendered with tools/build_report.py. Code listings are extracted from the actual Java sources, not manually copied examples. Figures come from tracked screenshots and rendered UML.

A delivery bundle includes the PDF, editable document, executable JAR and source ZIP. The ZIP excludes Git history, IDE settings, caches and generated target output. It includes Maven Wrapper, resources, tests and report regeneration sources. The delivery manifest records exact commit and artifact checksums. The team should review names, course-specific cover details and the final PDF before submitting to Moodle.